ZeroTraceFW: A Multi-Layered Cryptographic Storage and Policy Orchestration Framework

Abstract ZeroTraceFW is an advanced, multi-layered framework designed to orchestrate secure session management, cryptographic storage, and reactive policy enforcement. Developed with an emphasis on zero-knowledge persistence, ephemeral session handling, and autonomous trigger evaluation, the architecture aims to mitigate unauthorized data exposure through proactive and reactive secure deletion protocols. This paper details the structural design, data flow paradigms, cryptographic implementations, and operational characteristics of the ZeroTraceFW system.

---

1. Introduction

Modern environments demand robust systems capable of enforcing strict data access policies, ephemeral lifespans, and automated destruction upon unauthorized access attempts. ZeroTraceFW addresses these requirements through a synchronized, three-layered architecture that securely manages files from ingestion to destruction. It implements AES-256-CBC encryption combined with PBKDF2 key derivation and ties decryption explicitly to a runtime policy engine (TriggerEngine).

---

2. Three-Layer Architecture

The ZeroTraceFW architecture separates interface handling, policy orchestration, and cryptographic persistence into three distinct operational layers.

Layer 1: Interface

The topmost layer encompasses `main.py` and `zerotracefw/ui.py`. This layer is responsible for session lifecycle management, processing user commands (via terminal or queued JSON commands from a Windows GUI), and broadcasting real-time status telemetry to `.zerotracefw/status.json`.

Layer 2: Control and Policy

Comprising `auth.py`, `sync.py`, `triggers.py`, and `audit.py`, the middle layer acts as the orchestrator. It manages:

- **Authentication**: Tracking failed attempts, handling lockouts, and processing duress hashes.
- **Syncing**: Determining when and how the Virtual File System (VFS) state must reflect physical changes in the mount directory.
- **Trigger Evaluation**: Continuously evaluating global and file-specific constraints (e.g., Time-to-Live, Read Limits).
- **Audit Trails**: Irrefutably logging actions, access attempts, and triggered destructions into a serialized state.

Layer 3: Cryptography and Storage

The foundational layer is defined by `encryption.py`, `key_derivation.py`, `filesystem.py`, `container.py`, and `wipe.py`. It is responsible for the actual cryptographic primitives (AES/PBKDF2), managing encrypted file entries in memory, persisting the encrypted vault (`container.pkl`), and executing the DoD 5220.22-M style secure wiping algorithms.

---

3. Data Flow Paradigms

3.1. File Ingestion Path

When a user adds or edits a file in the active mount directory, the system processes it securely for vault ingestion.

1. **Detection**: `SyncEngine` scans the mount directory and detects the filesystem change.
2. **VFS Update**: The `VirtualFileSystem` initializes an `add_file` or `update_file` operation.
3. **Key Derivation**: `KeyDerivation` combines the master password with a uniquely generated 32-byte salt to derive a file-specific key via PBKDF2-HMAC-SHA256 (10,000 iterations).
4. **Encryption**: `EncryptionEngine` encrypts the raw file bytes using AES-256-CBC with a freshly generated 16-byte Initialization Vector (IV).
5. **Persistence**: `ContainerManager` serializes the updated VFS state, including the new ciphertext and metadata, out to `data/container.pkl`.

3.2. Trigger Destruction Path

The `TriggerEngine` operates continuously on each cycle of the runtime loop to enforce security policies.

1. **Global Checks**: Evaluates global triggers (e.g., dead-man's switch, global TTL). If fired, initiates the full vault destruction path.
2. **File Checks**: Iterates over file-specific metadata to evaluate limits:
    - *Time-to-Live (TTL) Expired* -> Destroy File
    - *Read Limit Exceeded* -> Destroy File
    - *Deadline Reached* -> Destroy File
3. **Audit**: `AuditLogger` records a `TRIGGER_FIRE` and subsequent `DESTRUCTION` event for post-incident analysis.

---

4. Cryptographic Implementation Details

ZeroTraceFW enforces encryption at the granular file-entry level, utilizing established cryptographic standards to ensure data at rest remains inaccessible without the correct authorization state.

- **Cipher Strategy**: AES-256-CBC
- **Key Derivation Function (KDF)**: PBKDF2-HMAC-SHA256

- **Iteration Count**: 10,000 iterations
- **Key Length**: 32 bytes (256-bit)
- **IV Length**: 16 bytes (128-bit, unique per encryption operation)
- **Salt Length**: 32 bytes (256-bit, unique per file entry)

The ciphertext, IV, Salt, Key metadata, and arbitrary tags are stored in the VFS memory structure and subsequently written to the serialized container payload. The master key itself is never serialized to disk and exists exclusively in volatile process memory during an unlocked session.

---

5. Secure Deletion and Sanitization Protocol

To thwart forensic recovery attempts on underlying storage media, ZeroTraceFW employs a rigorous multi-pass secure wiping protocol.

5.1. Granular File Destruction

1. Overwrite full file allocation with random bytes (Pass 1).
2. Overwrite full file allocation with different random bytes (Pass 2).
3. Overwrite full file allocation with different random bytes (Pass 3).
4. Overwrite full file allocation with zeros (Pass 4).
5. Truncate the file to 0 bytes.
6. Issue OS-level delete command.
7. Drop file entry from encrypted vault (VFS memory).
8. Overwrite and explicitly clear key, IV, and salt material within the process address space.

5.2. Total Vault Destruction (Wipe)

Triggered by extreme policy violations (e.g., Duress code entry, global TTL expiration, or max authentication failures):

1. Execute Granular File Destruction on every file in the active mount directory.
2. Securely wipe the `data/container.pkl` persistent storage.
3. Clear all Virtual File System (VFS) entries.
4. Force runtime garbage collection to clear remnant variables in heap memory.

---

6. Authentication and Threat Modeling

ZeroTraceFW models access as a high-friction, fail-secure gate. The auth state is persisted across reboots via the serialized container, retaining critical telemetry such as `failed_attempts`.

- **Standard Access**: Successful master password entry grants access and resets the `failed_attempts` counter.
- **Duress Mechanism**: Inputting a predefined duress password authenticates as a standard user superficially, but immediately triggers the Total Vault Destruction protocol, rendering the vault permanently inaccessible while providing a cover message.
- **Brute Force Mitigation**: Exceeding the predefined `max_attempts` immediately

transitions the state to `is_locked = True`. This executes a Total Vault Destruction and purges the authentication state.

---

7. Conclusion

ZeroTraceFW establishes a resilient and defensively aggressive paradigm for file management. By integrating a three-tiered architecture with reactive triggers and destructive secure-wiping protocols, it ensures that unauthorized access attempts, environmental compromise, or simple time expiration results in the permanent cryptological and physical sanitization of the secured data.